

Playwright

Complete Documentation Guide

Basic

.

Intermediate

.

Advanced

Browser Automation · End-to-End Testing · Web Scraping

Microsoft Playwright · JavaScript / TypeScript

Table of Contents

- | | |
|----|---|
| 1. | What is Playwright?
Overview & key features |
| 2. | Installation & Setup
Install, config, first test |
| 3. | Selectors & Locators
Finding elements the right way |
| 4. | Actions
Click, fill, type, upload and more |
| 5. | Assertions
Expect API and auto-waiting |
| 6. | Page Object Model
Structuring tests for real projects |
| 7. | Network Interception
Mocking APIs and saving auth state |
| 8. | Parallel Execution & CI
Config, workers, GitHub Actions |
| 9. | Quick Reference
Cheat sheet |

1. What is Playwright?

Playwright is a modern, open-source browser automation library developed by Microsoft. It lets you write code that controls a real browser — Chrome, Firefox, or Safari — to test web applications, automate repetitive tasks, or scrape data from websites.

■ Think of it as a robot that can open a browser, click buttons, fill forms, and verify that your website works correctly — all automatically.

Auto-waiting	Playwright automatically waits for elements to appear, be clickable, or become visible before acting on them. No more flaky <code>sleep()</code> calls.
Multi-browser	One unified API works across Chromium (Chrome/Edge), Firefox, and WebKit (Safari). Write once, test everywhere.
Multi-language	Supports JavaScript, TypeScript, Python, Java, and C#. The same concepts apply across all languages.
Debug tools	Built-in trace viewer, screenshots, and video recording on failure. The Playwright Inspector lets you step through tests visually.
Network control	Intercept, mock, and modify network requests. Perfect for testing error states and offline scenarios.
Mobile emulation	Emulate any mobile device with correct viewport, touch events, and device pixel ratio.

2. Installation & Setup

Step 1 — Create a new project

```
# Create a brand-new Playwright project (recommended for beginners)
npm init playwright@latest

# This will ask you:
# - TypeScript or JavaScript?
# - Where to put tests? (tests/)
# - Add GitHub Actions CI workflow? (yes)
# - Install browsers? (yes)
```

Step 2 — Or add to an existing project

```
npm install -D @playwright/test
npx playwright install # downloads Chromium, Firefox, WebKit
```

Project structure after setup

```
my-project/
├── tests/
│   └── example.spec.ts # your test files go here
├── playwright.config.ts # configuration
├── package.json
└── node_modules/
```

Your first test

```
// tests/example.spec.ts
import { test, expect } from '@playwright/test';

test('page has correct title', async ({ page }) => {
  await page.goto('https://example.com');
  await expect(page).toHaveTitle(/Example Domain/);
});

test('link is visible', async ({ page }) => {
  await page.goto('https://example.com');
  await expect(page.getByRole('link', { name: 'More information' })).toBeVisible();
});
```

Running tests

```
npx playwright test # run all tests headlessly  
npx playwright test --headed # watch the browser open  
npx playwright test --ui # open Playwright UI mode (best for learning!)  
npx playwright show-report # open HTML test report  
npx playwright codegen https://site # record your clicks as code
```

■ Pro tip: Use --ui mode when learning. It shows a timeline of every action, screenshots at each step, and lets you re-run tests with one click.

3. Selectors & Locators

Selectors tell Playwright *which element* to interact with. Always prefer user-facing locators — they are more readable, resilient to code changes, and better reflect how real users interact.

Recommended locators (use these first)

```
// By ARIA role — the most robust selector
page.getByRole('button', { name: 'Submit' })
page.getByRole('link', { name: 'Home' })
page.getByRole('textbox', { name: 'Search' })

// By form label
page.getByLabel('Email address')
page.getByLabel('Password')

// By placeholder text
page.getByPlaceholder('Enter your name')

// By visible text
page.getByText('Welcome back, Alice')
page.getByText('Welcome back, Alice', { exact: true })

// By test ID (add data-testid to your HTML for stable selectors)
page.getByTestId('login-button')
// HTML: <button data-testid='login-button'>Login</button>
```

CSS & XPath (use when needed)

```
page.locator('.nav-item > a')
page.locator('#main-content h2')
page.locator("//h2[contains(text(),'Hello')]")
```

Chaining & filtering

```
// Find a link only inside the sidebar
page.locator('.sidebar').getByRole('link', { name: 'Settings' })

// Filter a list of items to find the one with text "Apple"
page.getByRole('listitem').filter({ hasText: 'Apple' })

// Get the 3rd item (0-indexed)
page.getByRole('listitem').nth(2)

// First and last
page.getByRole('listitem').first()
page.getByRole('listitem').last()
```

■ Avoid CSS class selectors like `.btn-primary-blue` — they break whenever a developer renames a class. Use `getByRole` or `data-testid` attributes for stable, maintainable tests.

4. Actions

Click, fill, and type

```
// fill() – clears first, then types (use for inputs)
await page.getByLabel('Username').fill('alice');
await page.getByLabel('Password').fill('secret123');

// type() – types character by character (simulates real typing)
await page.getByLabel('Search').type('playwright');

// click
await page.getByRole('button', { name: 'Login' }).click();
await page.getByText('Terms of Service').click();

// double click and right click
await page.getByText('filename.txt').dblclick();
await page.getByText('item').click({ button: 'right' });
```

Forms

```
// Dropdown (select element)
await page.getByLabel('Country').selectOption('BD'); // by value
await page.getByLabel('Country').selectOption({ label: 'Bangladesh' }); // by label

// Checkbox and radio
await page.getByRole('checkbox', { name: 'Remember me' }).check();
await page.getByRole('checkbox').unchecked();
await page.getByRole('radio', { name: 'Express' }).check();

// File upload
await page.getByLabel('Profile photo').setInputFiles('photo.jpg');
await page.getByLabel('Documents').setInputFiles(['doc1.pdf', 'doc2.pdf']);
```

Navigation & keyboard

```
await page.goto('https://example.com');
await page.goto('/dashboard'); // works with baseURL set in config
await page.goBack();
await page.reload();

// Keyboard shortcuts
await page.keyboard.press('Enter');
await page.keyboard.press('Control+A'); // select all
await page.keyboard.press('Escape');

// Hover (useful for dropdown menus)
await page.getByText('Products').hover();
await page.getByRole('menuitem', { name: 'Pricing' }).click();
```

Waiting

```
// Playwright auto-waits, but sometimes you need explicit waits
await page.waitForURL('/dashboard'); // wait for navigation
await page.waitForSelector('.toast'); // wait for element
await page.getByRole('spinner').waitFor({ state: 'hidden' }); // wait to disappear
```

5. Assertions

Playwright's **expect()** assertions automatically retry until the condition is true or the timeout expires (default 5 seconds). This eliminates most race conditions without any manual waiting.

Page assertions

```
await expect(page).toHaveURL('https://example.com/dashboard');
await expect(page).toHaveURL(/dashboard/); // regex match
await expect(page).toHaveTitle('Dashboard');
await expect(page).toHaveTitle(/Dashboard/);
```

Element state assertions

```
const button = page.getByRole('button', { name: 'Submit' });

await expect(button).toBeVisible();
await expect(button).toBeHidden();
await expect(button).toBeEnabled();
await expect(button).toBeDisabled();
await expect(button).toBeChecked(); // for checkboxes
await expect(button).toBeFocused();
```

Text and value assertions

```
const heading = page.getByRole('heading', { level: 1 });
await expect(heading).toHaveText('Welcome, Alice');
await expect(heading).toContainText('Alice'); // partial match

const input = page.getByLabel('Email');
await expect(input).toHaveValue('alice@example.com');

// Count elements
await expect(page.getByRole('listitem')).toHaveCount(5);

// Check attribute
await expect(page.getByRole('link')).toHaveAttribute('href', '/home');
```

Visual regression (screenshot comparison)

```
// First run: creates a reference screenshot
// Subsequent runs: compares against the reference
await expect(page).toHaveScreenshot();
await expect(page.locator('.chart')).toHaveScreenshot('chart.png');

// Update snapshots when UI intentionally changes:
// npx playwright test --update-snapshots
```

6. Page Object Model

As your test suite grows, repeating the same locators and actions in every test becomes messy. The Page Object Model (POM) wraps each page of your app in a class, making tests cleaner and easier to maintain.

Create a page object

```
// pages/LoginPage.ts
import { Page, expect } from '@playwright/test';

export class LoginPage {
  readonly page: Page;

  constructor(page: Page) {
    this.page = page;
  }

  async goto() {
    await this.page.goto('/login');
  }

  async login(email: string, password: string) {
    await this.page.getByLabel('Email').fill(email);
    await this.page.getByLabel('Password').fill(password);
    await this.page.getByRole('button', { name: 'Login' }).click();
  }

  async expectErrorMessage(message: string) {
    await expect(this.page.getByRole('alert')).toContainText(message);
  }
}
```

Use the page object in tests

```
// tests/login.spec.ts
import { test, expect } from '@playwright/test';
import { LoginPage } from '../pages/LoginPage';

test('successful login', async ({ page }) => {
  const loginPage = new LoginPage(page);
  await loginPage.goto();
  await loginPage.login('alice@test.com', 'correctpass');
  await expect(page).toHaveURL('/dashboard');
});

test('wrong password shows error', async ({ page }) => {
  const loginPage = new LoginPage(page);
  await loginPage.goto();
  await loginPage.login('alice@test.com', 'wrongpass');
  await loginPage.expectErrorMessage('Invalid credentials');
});
```

- With Page Objects, if the login form changes, you only update LoginPage.ts — not every test that logs in.

7. Network Interception

Mock an API response

```
// Return fake data instead of hitting the real API
await page.route('**/api/users', route => {
  route.fulfill({
    status: 200,
    contentType: 'application/json',
    body: JSON.stringify([
      { id: 1, name: 'Alice' },
      { id: 2, name: 'Bob' }
    ]),
  });
});

await page.goto('/users');
await expect(page.getByText('Alice')).toBeVisible();
```

Block requests (speed up tests)

```
// Block images and fonts to make tests faster
await page.route('**/*.{png,jpg,gif,svg,woff2}', route => route.abort());

// Block third-party analytics
await page.route('**/google-analytics.com/**', route => route.abort());
```

Wait for a specific request

```
// Capture the login API response
const responsePromise = page.waitForResponse('**/api/login');
await page.getByRole('button', { name: 'Login' }).click();
const response = await responsePromise;

console.log(await response.json()); // { token: '...', user: {...} }
expect(response.status()).toBe(200);
```

Save and reuse login state

Instead of logging in before every test (slow!), log in once and save the session cookies. All tests then start already logged in.

```
// global-setup.ts – run once before all tests
import { chromium } from '@playwright/test';

async function globalSetup() {
  const browser = await chromium.launch();
  const page = await browser.newPage();
  await page.goto('http://localhost:3000/login');
  await page.getByLabel('Email').fill('admin@test.com');
  await page.getByLabel('Password').fill('password');
  await page.getByRole('button', { name: 'Login' }).click();
  await page.context().storageState({ path: 'auth.json' }); // save cookies
  await browser.close();
}

export default globalSetup;

// playwright.config.ts
export default defineConfig({
  globalSetup: './global-setup',
  use: { storageState: 'auth.json' }, // every test starts logged in
});
```

8. Parallel Execution & CI

playwright.config.ts — full configuration

```
import { defineConfig, devices } from '@playwright/test';

export default defineConfig({
  testDir: './tests',
  fullyParallel: true, // run all tests in parallel
  forbidOnly: !!process.env.CI, // fail if test.only is committed
  retries: process.env.CI ? 2 : 0, // retry failing tests on CI
  workers: process.env.CI ? 4 : undefined,
  reporter: [['html'], ['github']],
  use: {
    baseURL: 'http://localhost:3000',
    trace: 'on-first-retry', // record trace when test fails
    screenshot: 'only-on-failure',
    video: 'retain-on-failure',
    actionTimeout: 10_000, // 10s per action
    navigationTimeout: 30_000, // 30s for page loads
  },
  projects: [
    { name: 'chromium', use: { ...devices['Desktop Chrome'] } },
    { name: 'firefox', use: { ...devices['Desktop Firefox'] } },
    { name: 'webkit', use: { ...devices['Desktop Safari'] } },
    { name: 'mobile', use: { ...devices['Pixel 5'] } },
    { name: 'tablet', use: { ...devices['iPad Pro'] } },
  ],
});
```

GitHub Actions workflow

```
# .github/workflows/playwright.yml
name: Playwright Tests
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
    with: { node-version: 20 }
      - run: npm ci
      - run: npx playwright install --with-deps
      - run: npx playwright test
      - uses: actions/upload-artifact@v4
    if: always() # upload even when tests fail
    with:
      name: playwright-report
      path: playwright-report/
      retention-days: 30
```

Sharding — split tests across machines

```
# Run shard 1 of 3 on machine A
npx playwright test --shard=1/3

# Run shard 2 of 3 on machine B
npx playwright test --shard=2/3

# Merge reports from all shards
npx playwright merge-reports ./all-blobs --reporter html
```

9. Quick Reference Cheat Sheet

Locators

Action / Selector	Code
By role	<code>page.getByRole('button', { name: 'OK' })</code>
By label	<code>page.getByLabel('Email')</code>
By placeholder	<code>page.getByPlaceholder('Search')</code>
By text	<code>page.getByText('Welcome')</code>
By test ID	<code>page.getByTestId('submit-btn')</code>
CSS selector	<code>page.locator('.card > h2')</code>
nth item	<code>page.getByRole('row').nth(2)</code>
Filter	<code>page.getByRole('listitem').filter({ hasText: 'Apple' })</code>

Actions

Action / Selector	Code
Navigate	<code>await page.goto('/login')</code>
Fill input	<code>await page.getByLabel('Name').fill('Alice')</code>
Click	<code>await page.getByRole('button', { name: 'Submit' }).click()</code>
Select option	<code>await page.getByLabel('Size').selectOption('Large')</code>
Check box	<code>await page.getByRole('checkbox').check()</code>
Upload file	<code>await page.getByLabel('Photo').setInputFiles('img.jpg')</code>
Press key	<code>await page.keyboard.press('Enter')</code>
Hover	<code>await page.getByText('Menu').hover()</code>

Assertions

Action / Selector	Code
Has URL	<code>await expect(page).toHaveURL('/home')</code>
Has title	<code>await expect(page).toHaveTitle('Home')</code>
Is visible	<code>await expect(btn).toBeVisible()</code>
Is enabled	<code>await expect(btn).toBeEnabled()</code>

Has text	<code>await expect(el).toHaveText('Hello')</code>
Has value	<code>await expect(input).toHaveValue('alice@x.com')</code>
Count	<code>await expect(page.getByRole('row')).toHaveCount(10)</code>
Screenshot	<code>await expect(page).toHaveScreenshot()</code>

CLI Commands

Action / Selector	Code
Run all tests	<code>npx playwright test</code>
Run headed	<code>npx playwright test --headed</code>
Open UI mode	<code>npx playwright test --ui</code>
Record test	<code>npx playwright codegen https://site.com</code>
Show report	<code>npx playwright show-report</code>
Debug one test	<code>npx playwright test --debug login.spec.ts</code>
Update snapshots	<code>npx playwright test --update-snapshots</code>
Run one browser	<code>npx playwright test --project=chromium</code>